# Лабораторная работа 2

Счётчик калорий

## 1 Диаграмма классов

Диаграмма классов описывает программную структуру системы «Счётчик калорий», по которой написан код на C#. Имена классов, полей и операций записаны на английском языке, поэтому код совпадает с диаграммой буквально. Классы соответствуют сущностям модели предметной области: каталог продуктов (ProductCatalog), дневник питания (FoodDiary), порция (Portion), продукт (Product), ингредиент (Ingredient) и приготовленная партия (CookedBatch). Сценарий учёта организует обработчик учёта порции (PortionHandler). Сущность «Пользователь» классом не стала: программа ведёт один дневник, поэтому хранить пользователя не требуется.

Продукт и приготовленная партия обобщены абстрактным классом еды (Food). Еда имеет название и защищённый конструктор, который вызывают только наследники. Абстрактная операция GetKcalPer100g возвращает калорийность на 100 г: продукт берёт её с этикетки, а партия вычисляет по общим калориям и готовой массе. Операция CalculateKcal одна для всех видов еды и пересчитывает калорийность на заданную массу. Операция CanServe проверяет, можно ли подать порцию заданной массы; партия переопределяет её, чтобы масса порции не превышала готовую массу. Благодаря этому порцию можно взять и из партии, и непосредственно из продукта, а отдельный калькулятор калорий, показанный на диаграмме коммуникаций, при проектировании не потребовался: расчёт распределён между классами еды.

На диаграмме использованы пять видов связей. Направленные ассоциации показывают, какие объекты знает каждый класс: обработчик связан с каталогом, дневником и текущим предпросмотром порции, порция — со своей едой, ингредиент — с продуктом. Агрегации связывают каталог с продуктами и дневник с порциями: части существуют независимо от целого. Композиция связывает партию с ингредиентами, которые создаются внутри партии и без неё не существуют. Обобщение связывает продукт и партию с едой. Зависимости показывают классы, которые используются только как параметры или локальные переменные операций; партию обработчик создаёт сам, что отмечено стереотипом «create».

Калории фиксируются при создании объектов: общие калории партии накапливаются при добавлении ингредиентов, калории порции рассчитываются в её конструкторе, и последующие изменения каталога их не меняют. Операция Preview может

не сформировать порцию, поэтому у её результата указана кратность [0..1]. У каталога, обработчика и дневника нет атрибутов, поэтому раздел атрибутов у них не изображается. Классы, их члены и связи представлены на рисунке 1, описание диаграммы приведено в таблице 1.

Таблица 1 – Описание диаграммы классов

| Поле | Описание |
|---|---|
| Вопрос | Из каких классов состоит программа учёта порций и как эти классы связаны? |
| Читатель | Разработчик |
| Что сделает читатель | Напишет каркас кода: классы, поля, свойства, конструкторы и заголовки операций |
| Что показано | – классы с атрибутами и операциями, их видимостью и типами — этого достаточно для каркаса кода;<br>– абстрактный класс Food с двумя наследниками — порция одинаково рассчитывается из продукта и из партии;<br>– виды связей и кратности — они определяют поля-ссылки и коллекции;<br>– зависимости — классы, которые нужны только внутри операций. |
| Что не показано | – порядок вызовов внутри операций — он показан на диаграммах последовательностей;<br>– хранение данных между запусками программы;<br>– пользовательский интерфейс и обработка ошибок ввода. |
| Допущения | – формулы расчёта калорий на диаграмме не записаны и берутся из предметной области;<br>– типы Real, String, Boolean и DateTime реализуются типами double, string, bool и DateTime, а набор пар (String, Real) [1..*] — словарём Dictionary<string, double>;<br>– роль на конце ассоциации становится закрытым полем, а кратность 0..* или 1..* — списком List;<br>– кратность [0..1] у результата Preview реализуется типом Portion?, допускающим значение null. |

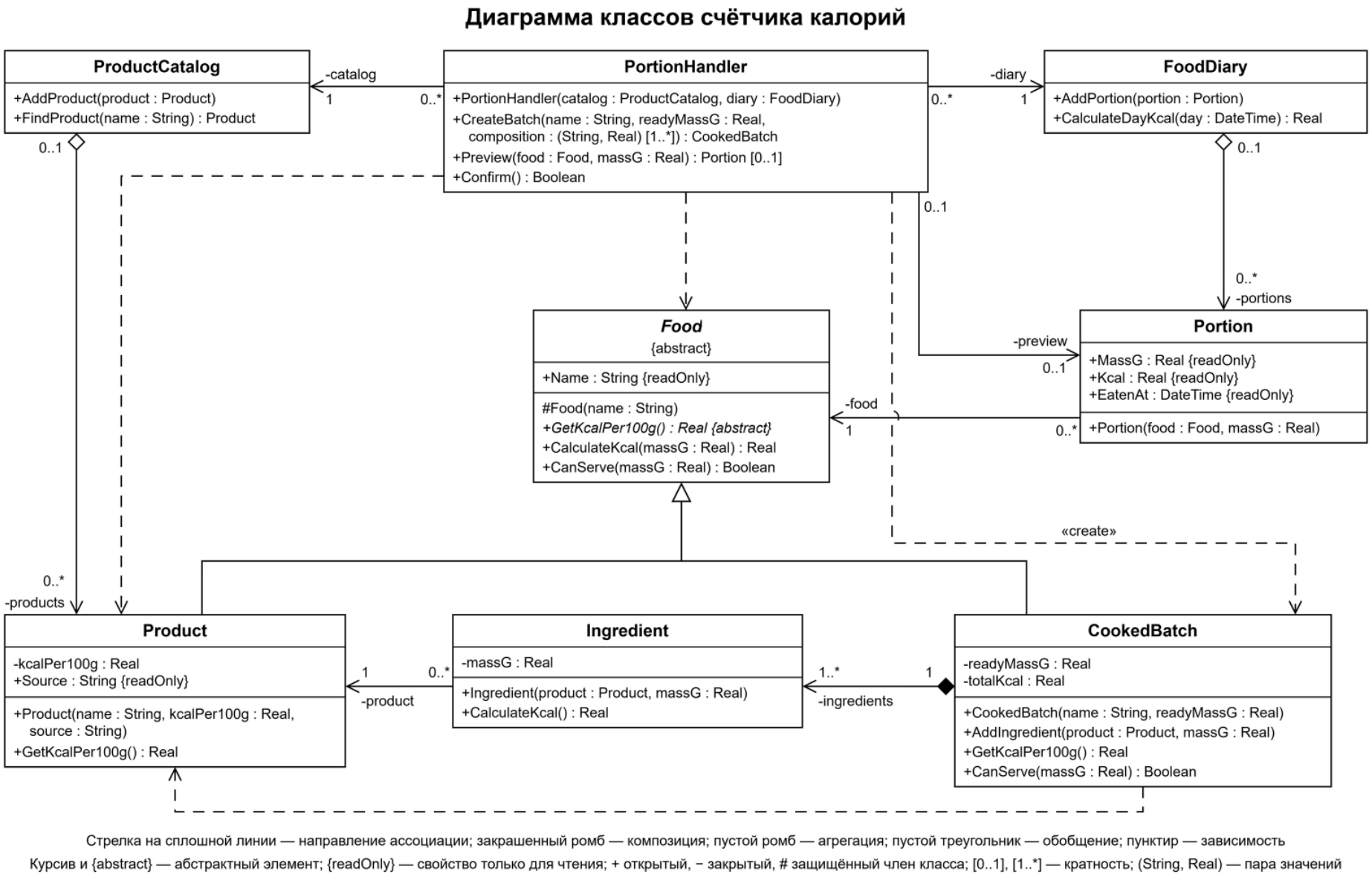



Рисунок 1 – Диаграмма классов счётчика калорий

## 2 Диаграмма последовательностей операции CreateBatch

Диаграмма описывает операцию CreateBatch обработчика учёта порции, которая создаёт приготовленную партию по названию, готовой массе и составу. Состав — набор пар (String, Real) [1..*]: название продукта из каталога и масса в граммах; в коде ему соответствует словарь Dictionary<string, double>. Участвуют обработчик, каталог catalog и создаваемая партия batch. Диаграмма демонстрирует фрейм цикла.

Сначала обработчик создаёт пустую партию сообщением new(name, readyMassG): блок партии стоит ниже остальных, так как объект появляется в этот момент. Затем во фрейме loop для каждой пары (productName, massG) обработчик находит продукт в каталоге, сохраняет его в переменную product и передаёт партии вместе с массой операцией AddIngredient (рисунок 4). Она результата не возвращает, поэтому возврат показан пунктиром без подписи. После цикла партия возвращается вызывающему объекту. На рисунке 2 показано приготовление каши из учебного примера: цикл выполняется дважды — для 40 г хлопьев и для 200 г молока. Описание диаграммы приведено в таблице 2.

Таблица 2 – Описание диаграммы последовательностей операции CreateBatch

| Поле | Описание |
|---|---|
| Вопрос | Как обработчик собирает приготовленную партию из введённого состава? |
| Читатель | Разработчик |
| Что сделает читатель | Напишет тело операции CreateBatch |
| Что показано | – партия создаётся до цикла, и ингредиенты добавляются в уже существующий объект;<br>– фрейм loop по составу — число ингредиентов заранее неизвестно;<br>– продукт берётся из каталога, поэтому партия получает объект продукта из каталога, а не его копию;<br>– созданная партия возвращается вызывающему объекту. |
| Что не показано | – что происходит внутри AddIngredient — это показано на рисунке 4;<br>– как пользователь вводит состав и массы;<br>– реакция на пустой состав. |
| Допущения | – если названия нет в каталоге, FindProduct выбрасывает исключение KeyNotFoundException; на диаграмме это не изображено, так как продукты выбираются из каталога и такая ситуация считается ошибкой программы;<br>– набор пар в коде — словарь Dictionary<string, double>, цикл — оператор foreach; названия не повторяются, так как ключи словаря уникальны, хотя кратность этого не выражает;<br>– кратность 1..* у ингредиентов операция не проверяет: предполагается, что состав непустой. |

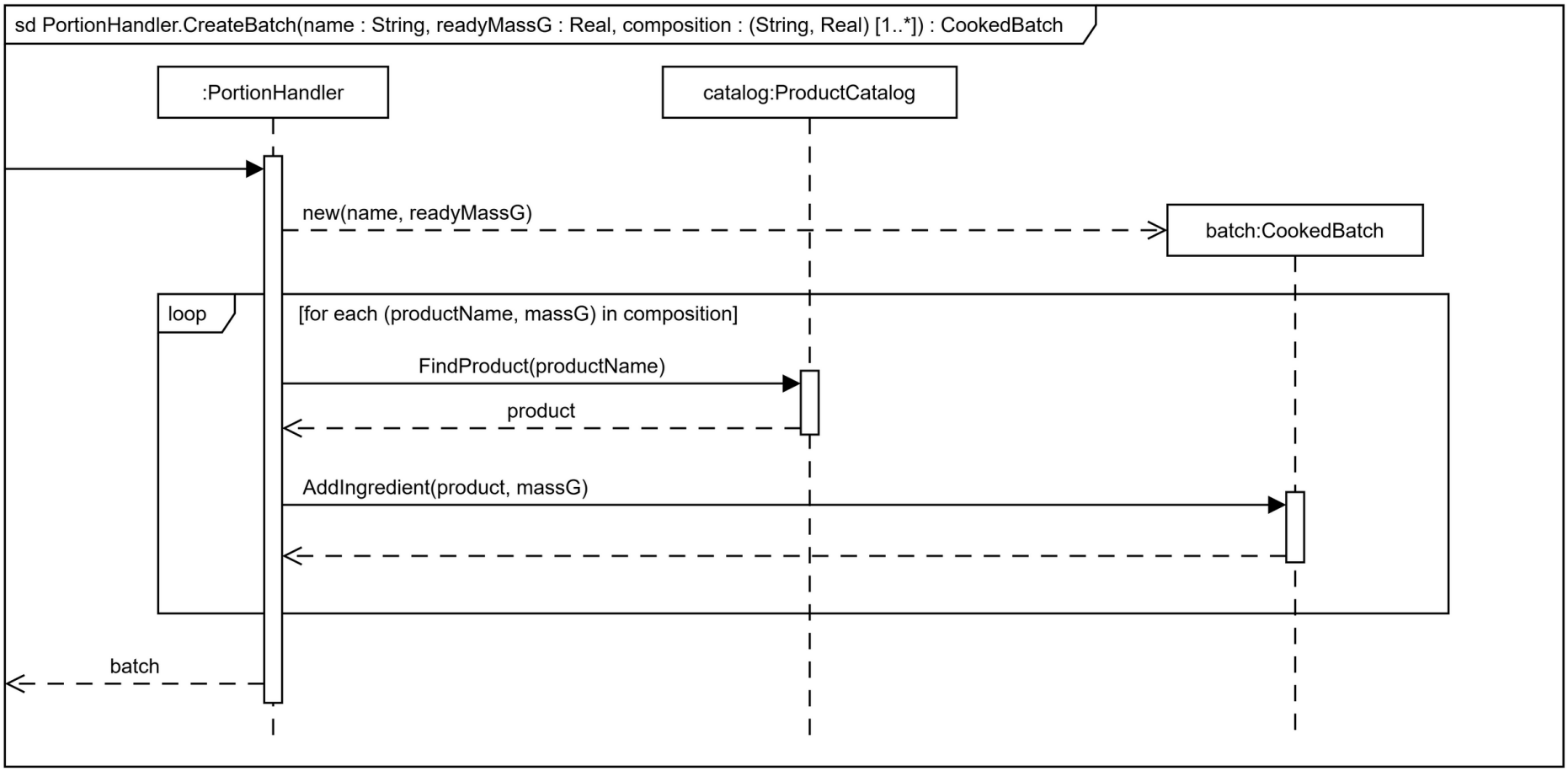



Рисунок 2 – Диаграмма последовательностей при приготовлении каши из хлопьев и молока

## 3 Диаграмма последовательностей операции Preview

Диаграмма описывает операцию Preview обработчика учёта порции: предпросмотр порции заданной массы из еды — продукта или приготовленной партии. Участвуют обработчик, еда food и создаваемая порция preview. Участник food имеет абстрактный тип Food, поэтому реализация GetKcalPer100g выбирается во время работы программы. Диаграмма демонстрирует вложенные активности.

Обработчик спрашивает у еды, можно ли подать порцию такой массы, и сохраняет ответ в canServe. Если можно, создаётся порция: её конструктор вызывает у еды CalculateKcal(massG), а та — собственную операцию GetKcalPer100g, самовызов показан петлёй. Активности вложены: CalculateKcal выполняется во время работы конструктора порции, а конструктор — во время Preview. Калории сохраняются в свойство Kcal, а порция возвращается обработчику в поле preview. В ветви [else] предпросмотр сбрасывается в null, чтобы прежний предпросмотр нельзя было подтвердить.

На рисунке 3 показан предпросмотр порции каши: 110 г дают 122,4 ккал, а 300 г больше готовой массы 220 г, и операция возвращает null. Описание диаграммы приведено в таблице 3.

Таблица 3 – Описание диаграммы последовательностей операции Preview

| Поле | Описание |
|---|---|
| Вопрос | Как обработчик формирует предпросмотр порции и рассчитывает её калории? |
| Читатель | Разработчик |
| Что сделает читатель | Напишет операцию Preview, конструктор порции и операцию CalculateKcal класса Food |
| Что показано | – проверка CanServe выполняется до создания порции, поэтому порция с недопустимой массой не появляется;<br>– вложенные активности Preview → конструктор порции → CalculateKcal: калории рассчитываются один раз при создании порции;<br>– ветвь [else] сбрасывает прежний предпросмотр. |
| Что не показано | – правила CanServe для продукта и для партии;<br>– подтверждение предпросмотра и запись порции в дневник (операция Confirm);<br>– вывод предпросмотра пользователю. |
| Допущения | – формула калорий «калорийность на 100 г × масса / 100» на диаграмме не изображена;<br>– конкретный объект food — продукт или партия; на диаграмме указан их общий абстрактный тип;<br>– кратность [0..1] у результата реализуется типом Portion?. |

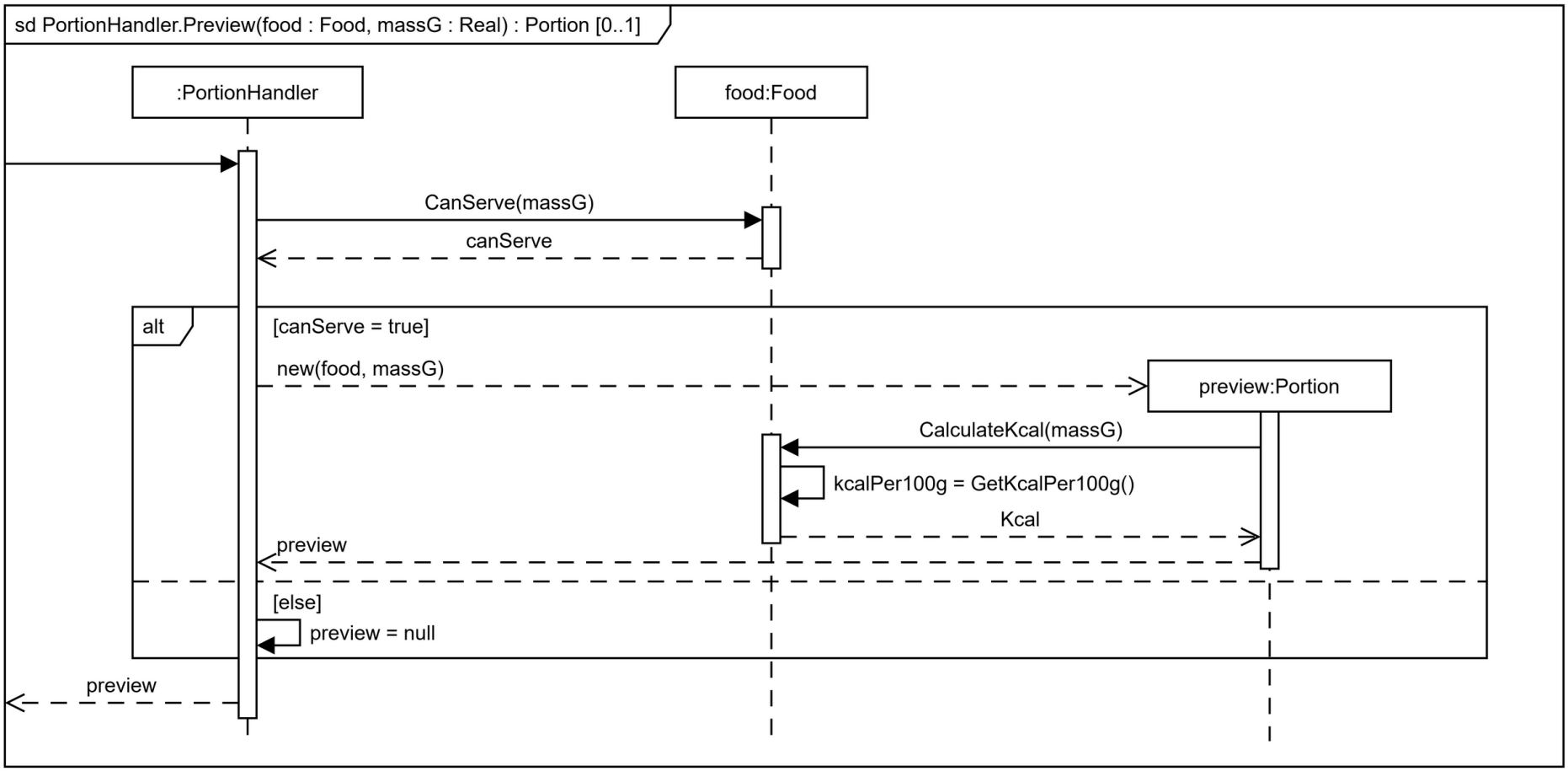



Рисунок 3 – Диаграмма последовательностей при предпросмотре порции каши

# 4 Диаграмма последовательностей операции AddIngredient

Диаграмма описывает операцию AddIngredient приготовленной партии, которая добавляет в партию ингредиент — массу конкретного продукта. Во взаимодействии участвуют партия, создаваемый ингредиент ingredient и продукт product. Диаграмма демонстрирует создание объекта.

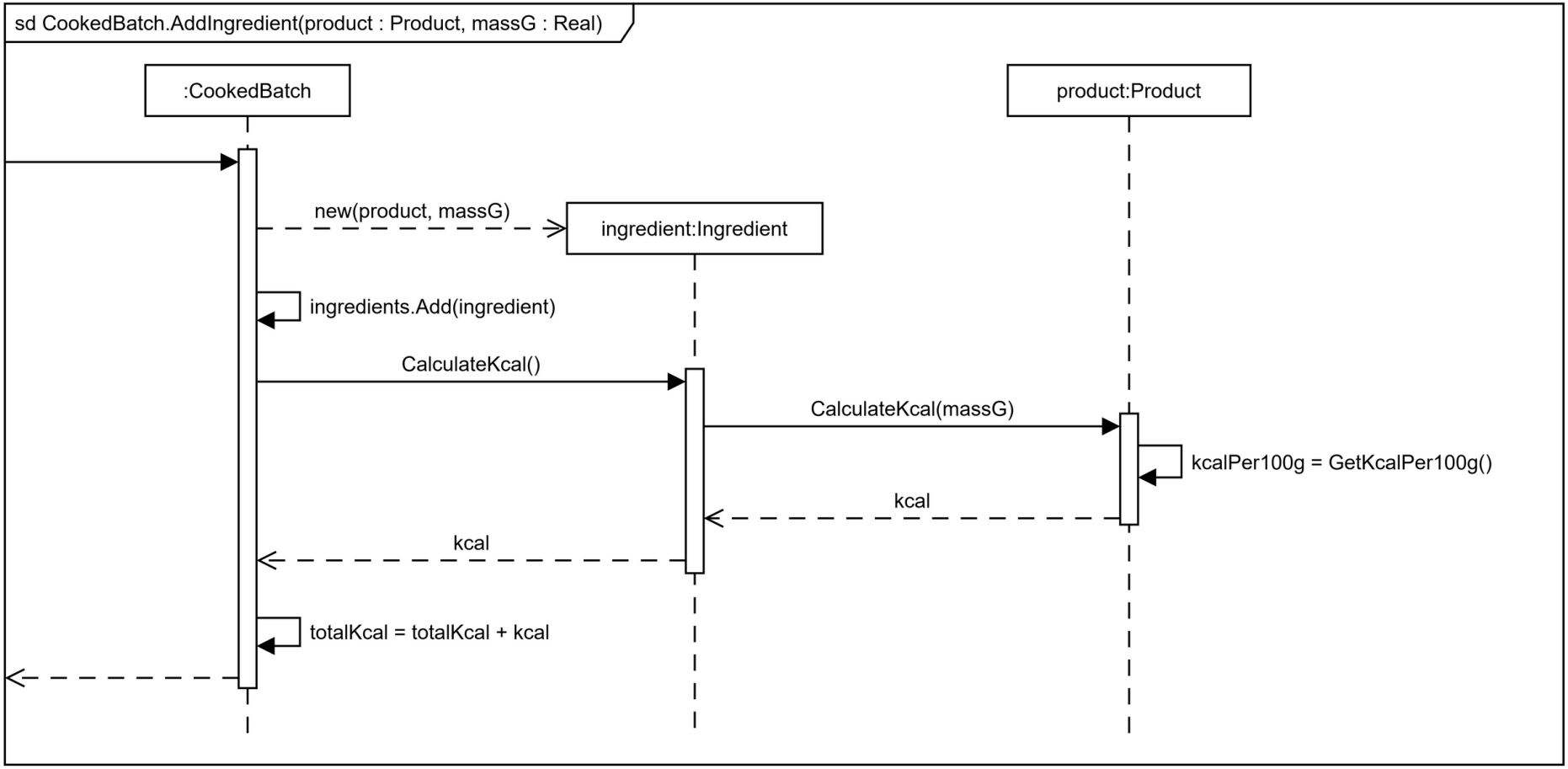
Партия создаёт ингредиент сообщением new(product, massG): ингредиент является частью партии по композиции, поэтому его создаёт сама партия. Созданный ингредиент добавляется в список ingredients. Затем партия запрашивает у ингредиента калории. Ингредиент вызывает у продукта операцию CalculateKcal(massG), а продукт — собственную операцию GetKcalPer100g. Результат возвращается в переменную kcal и прибавляется к общим калориям партии totalKcal. Самовызов GetKcalPer100g и действия внутри операции изображены петлями. Операция результата не возвращает, поэтому по её завершении управление возвращается вызывающему объекту пунктиром без подписи.

Общие калории партии накапливаются при добавлении ингредиентов и далее не пересчитываются. На рисунке 4 показано добавление молока в кашу: 200 г молока дают 104 ккал, и вместе с хлопьями общие калории каши составляют 140,8 + 104 = 244,8 ккал. Описание диаграммы приведено в таблице 4.

Таблица 4 – Описание диаграммы последовательностей операции AddIngredient

| Поле | Описание |
|---|---|
| Вопрос | Как партия добавляет ингредиент и учитывает его калории? |
| Читатель | Разработчик |
| Что сделает читатель | Напишет операции AddIngredient и CalculateKcal класса Ingredient |
| Что показано | – создание ингредиента внутри партии — следствие композиции;<br>– добавление ингредиента в список партии;<br>– расчёт калорий по цепочке партия → ингредиент → продукт;<br>– накопление общих калорий в поле totalKcal. |
| Что не показано | – пересчёт общих калорий на 100 г готовой массы (операция GetKcalPer100g партии);<br>– удаление и изменение ингредиентов — программа их не поддерживает;<br>– проверка массы ингредиента. |
| Допущения | – петли ingredients.Add(ingredient) и totalKcal = totalKcal + kcal обозначают операторы внутри метода, а не вызовы операций;<br>– возврат из операции без результата (void) показан пунктиром без подписи;<br>– формула калорий продукта не изображена. |

## Добавление молока в кашу




Сплошная стрелка — вызов операции; пунктирная — возврат (подписан переменной, куда сохраняется результат) или создание объекта new(...)

Узкий прямоугольник — активность; петля — самовызов или действие внутри операции; пунктир без подписи — возврат без результата

Рисунок 4 – Диаграмма последовательностей при добавлении молока в кашу

## 5 Диаграмма объектов

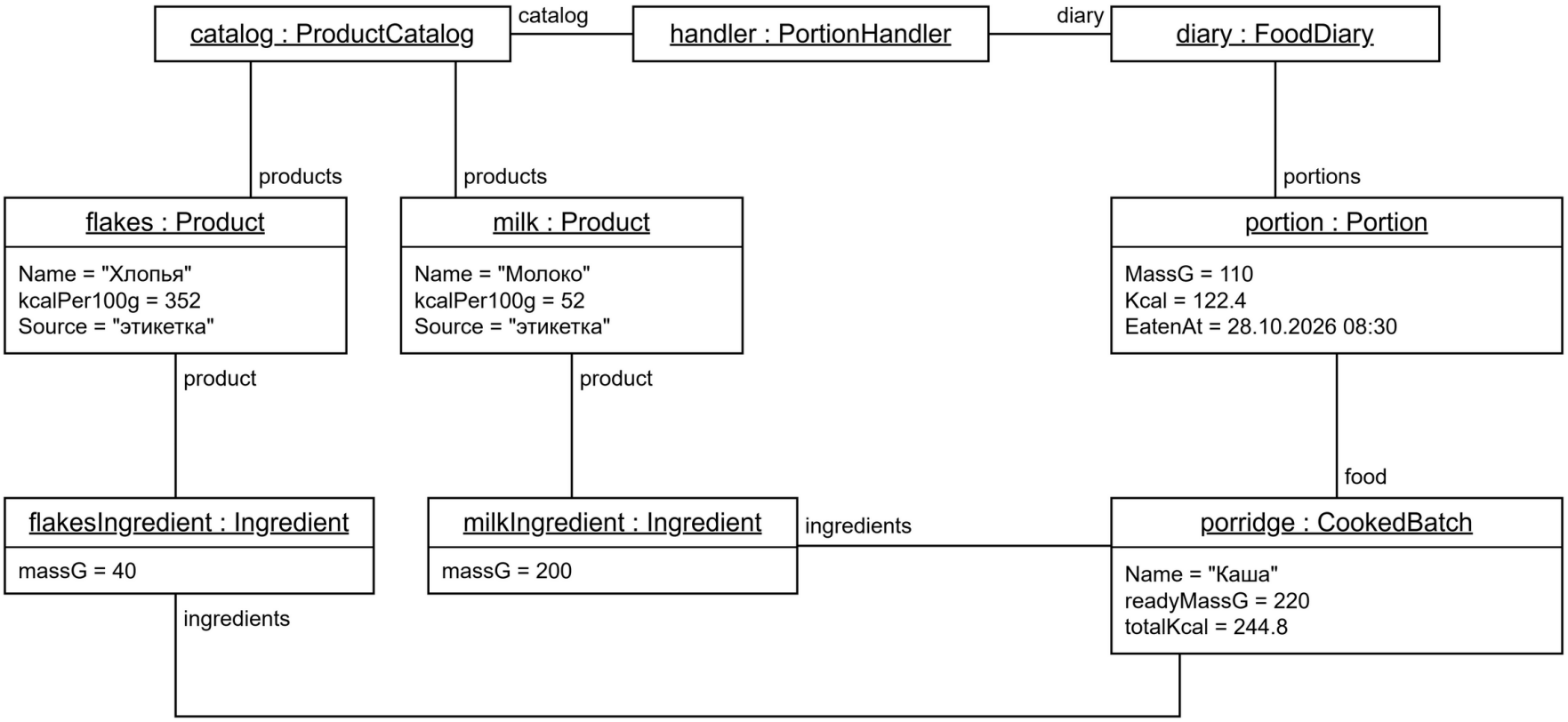
Диаграмма объектов показывает состояние программы в учебном примере после подтверждения порции. Такое же состояние получается, если ввести учебный пример в программе. Каталог содержит продукты «Хлопья» (352 ккал на 100 г) и «Молоко» (52 ккал на 100 г). Каша содержит два ингредиента — 40 г хлопьев и 200 г молока, её готовая масса 220 г, общие калории 244,8 ккал. Дневник содержит порцию каши массой 110 г и калорийностью 122,4 ккал. Обработчик связан с каталогом и дневником.

Имена объектов подчёркнуты и записаны в виде «имя : Класс», под чертой указаны значения полей, включая унаследованное поле Name. Связи являются экземплярами ассоциаций, агрегаций и композиции диаграммы классов; у их концов подписаны роли. Диаграмма показывает значимую связь: ингредиенты ссылаются на те же объекты продуктов, что хранятся в каталоге, а порция — на партию, а не на продукты. Связи обработчика с предпросмотром нет, так как после подтверждения предпросмотр сброшен. На рисунке 5 показаны каша из хлопьев и молока и её порция в дневнике, описание диаграммы приведено в таблице 5.

Таблица 5 – Описание диаграммы объектов

| Поле | Описание |
|---|---|
| Вопрос | Какие объекты и связи существуют после записи порции каши в дневник? |
| Читатель | Тестировщик |
| Что сделает читатель | Введёт учебный пример в программе и сверит значения полей и связи объектов при отладке с ожидаемыми |
| Что показано | – значения полей учебного примера — эталон для проверки расчётов;<br>– общие объекты продуктов у каталога и ингредиентов;<br>– связь порции с партией, а не с продуктами;<br>– отсутствие связи с предпросмотром после подтверждения. |
| Что не показано | – промежуточные состояния, например до подтверждения, когда порция связана с обработчиком, а дневник пуст;<br>– порции за другие дни;<br>– иерархия наследования классов. |
| Допущения | – время EatenAt условное: в программе оно берётся из часов компьютера в момент создания порции;<br>– дробные значения записаны с точкой, как в коде. |

## Каша из хлопьев и молока и её порция в дневнике




Подчёркнутое имя — объект в виде имя : Класс; под чертой — значения его полей

Линия — связь между объектами; у конца — роль, то есть поле, через которое объект знает соседа

Состояние после Confirm(): порция уже в дневнике, поэтому связи handler с preview нет

Рисунок 5 – Диаграмма объектов после записи порции каши в дневник

## 6 Программный код

Код написан на языке C# для платформы .NET 10 и лежит в папке Counter. Решение CalorieCounter.slnx содержит консольный проект CalorieCounter: классы диаграммы классов находятся в нём, каждый в отдельном файле пространства имён CalorieCounter.

Код получен прямым проектированием по диаграммам: каждому элементу диаграммы соответствует определённая конструкция языка. Соответствие приведено в таблице 6.

Таблица 6 – Соответствие элементов диаграмм и кода

| **Элемент диаграммы** | **Конструкция C#** |
|---|---|
| Закрытый атрибут -massG : Real | закрытое поле private double massG |
| Свойство +Name : String {readOnly} | свойство только для чтения public string Name { get; } |
| Роль на конце ассоциации с кратностью 1 или 0..1 | поле-ссылка; при 0..1 — тип, допускающий null |
| Роль с кратностью 0..* или 1..* | поле-список List<T> |
| Набор пар (String, Real) [1..*] | словарь Dictionary<string, double> |
| Защищённый конструктор #Food(name : String) | конструктор protected Food(string name) |
| Абстрактная операция | абстрактный метод без тела |
| Операция, повторённая в наследнике | метод virtual в предке и override в наследнике |
| Обобщение | наследование class Product : Food с вызовом конструктора предка : base(name) |
| Зависимость | параметр или локальная переменная операции |
| Сообщение M(x) с возвратом r | вызов с сохранением результата: Тип r = obj.M(x); |
| Создание объекта new(a, b) | T obj = new T(a, b); |
| Фрейм loop | цикл foreach |
| Фрейм alt | условный оператор if … else |

Метод Main запускает консольное меню: пользователь вносит в каталог продукты с калорийностью с этикетки, готовит из них блюдо (операция CreateBatch), записывает порцию продукта или блюда в два шага — предпросмотр (Preview) и подтверждение (Confirm) — и смотрит итог за день (CalculateDayKcal). Для учебного примера программа выводит 111,3 ккал на 100 г каши и 122,4 ккал для порции 110 г, а порцию 300 г отклоняет.